

UNIT-6

Windows Forms and ADO.NET

Introduction –

Windows Forms is a **Graphical User Interface (GUI)** class library which is bundled in .Net Framework.

Its main purpose is to provide an easier interface to develop the applications for desktop, tablet, PCs. It is also termed as the **WinForms**.

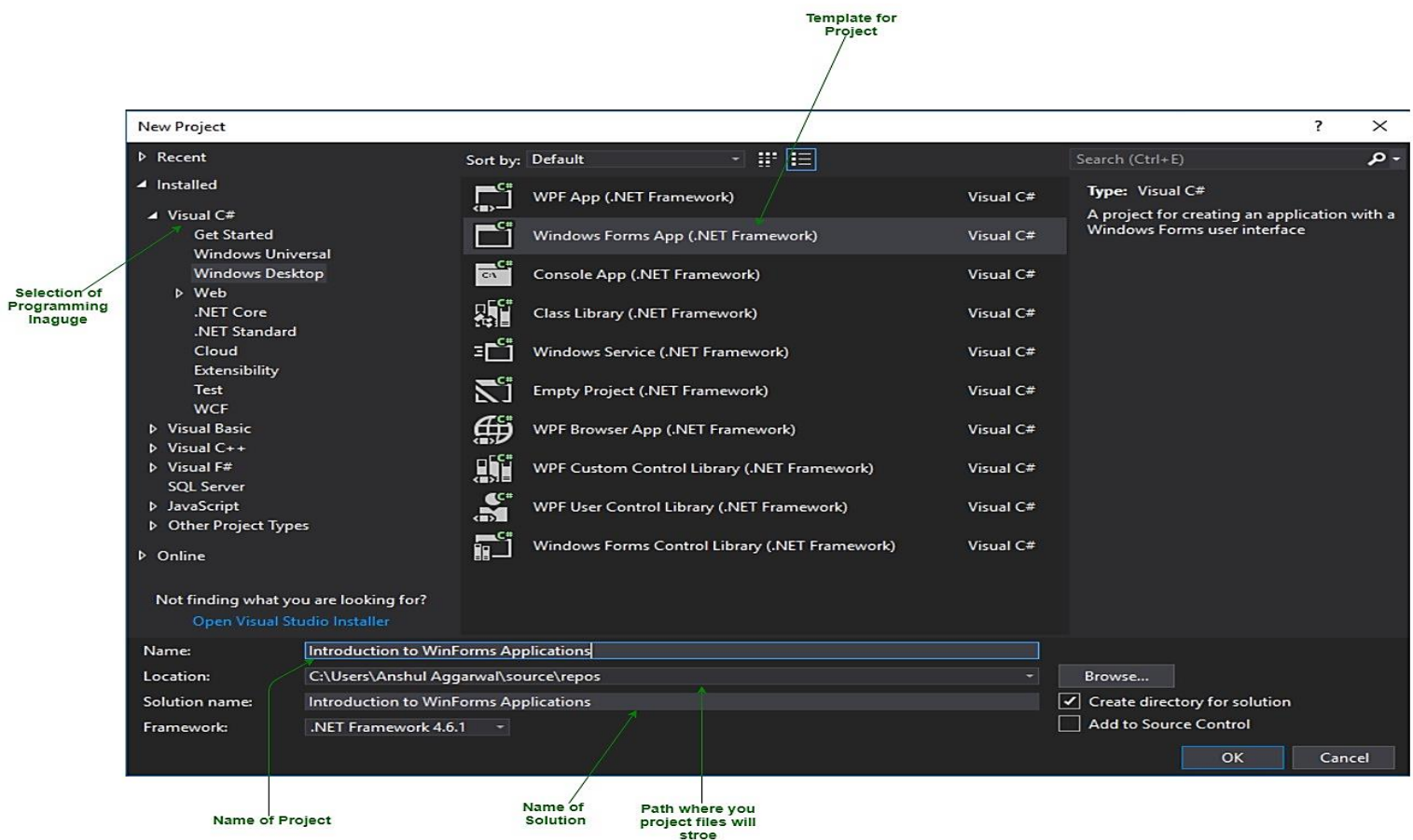
The applications which are developed by using Windows Forms or WinForms are known as the **Windows Forms Applications** that runs on the desktop computer.

WinForms can be used only to develop the Windows Forms Applications not web applications.

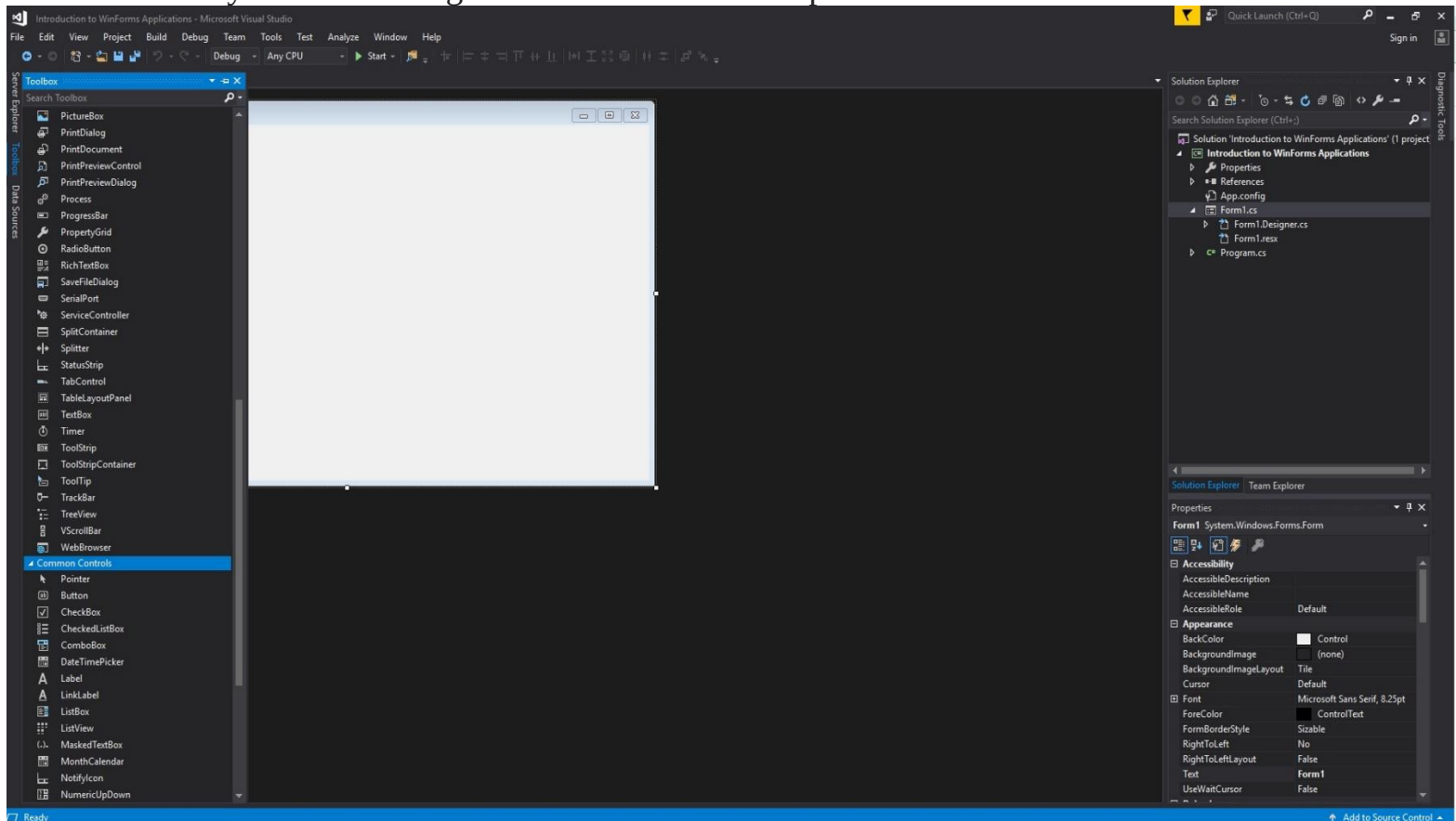
WinForms applications can contain the different type of controls like labels, textbox, list boxes, tooltip etc.

Creating a Windows Forms Application

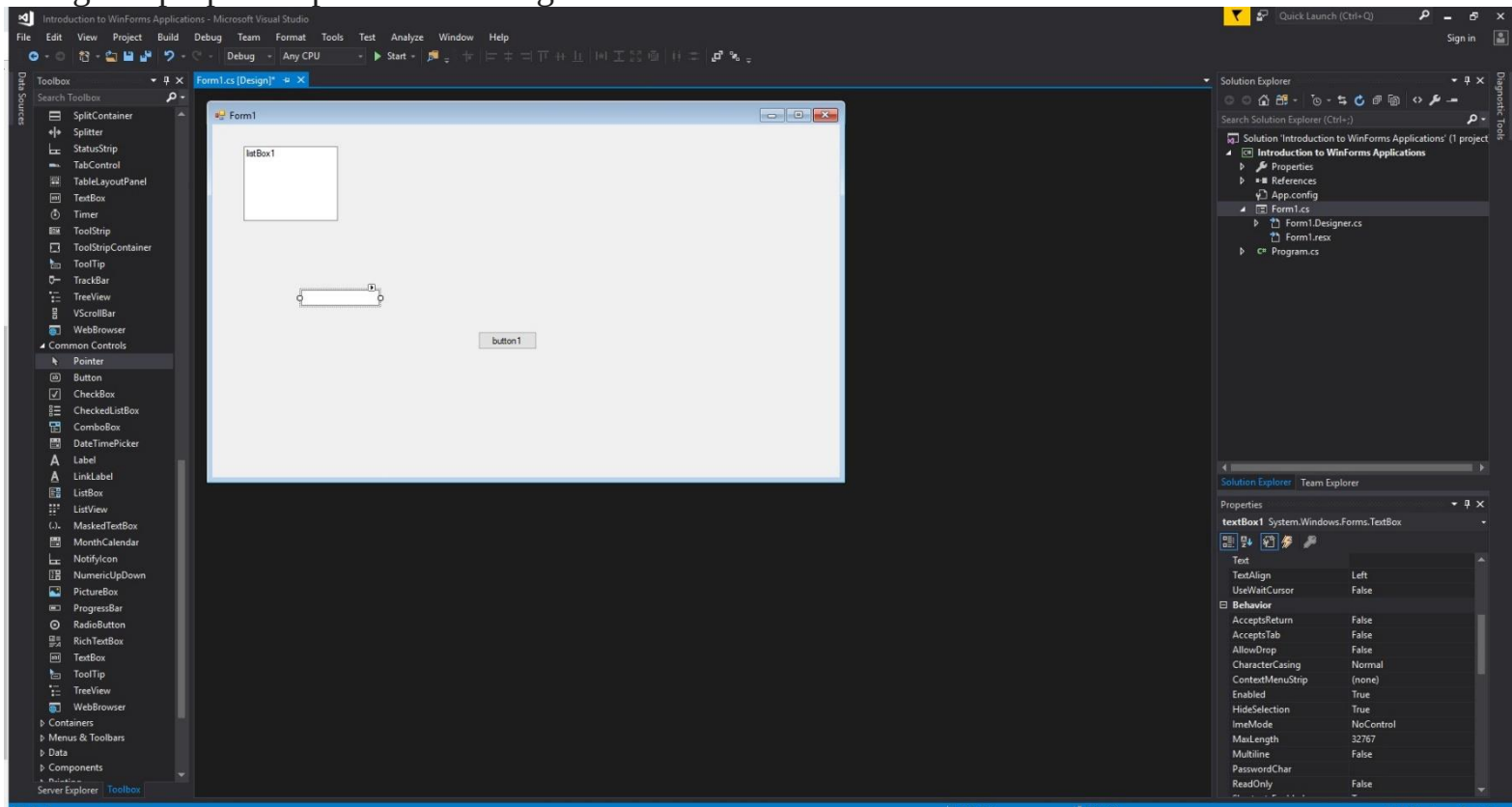
First, open the Visual Studio then Go to File -> New -> Project to create a new project and then select the language as *Visual C#* from the left menu. Click on *Windows Forms App(.NET Framework)* in the middle of current window. After that give the project name and Click OK.



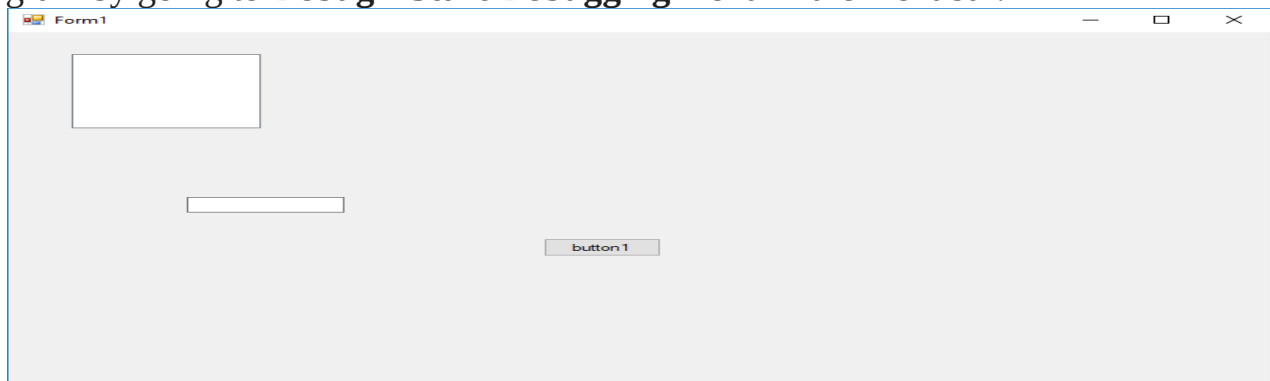
- You can also reset the window layout by setting it to default. To set the default layout, go to **Window -> Reset Window Layout** in Visual Studio Menu.
- Now to add the controls to your WinForms application go to **Toolbox** tab present in the extreme left side of Visual Studio. Here, you can see a list of controls. To access the most commonly used controls go to **Common Controls** present in Toolbox tab.



Now drag and drop the controls that you needed on created Form. For example, if you can add TextBox, ListBox, Button etc. as shown below. By clicking on the particular dropped control you can see and change its properties present in the right most corner of Visual Studio.



- In the above image, you can see the TextBox is selected and its properties like TextAlign, MaxLength etc. are opened in right most corner. You can change its properties' values as per the application need. The code of controls will be automatically added in the background. You can check the *Form1.Designer.cs* file present in the Solution Explorer Window.
- To run the program you can use an **F5 key** or **Play button** present in the toolbar of Visual Studio. To stop the program you can use pause button present in the ToolBar. You can also run the program by going to **Debug->Start Debugging** menu in the menubar.



ADO.NET

ADO.NET is a set of classes (a framework) to interact with data sources such as databases and XML files.

ADO is the acronym for ActiveX Data Objects.

It allows us to connect to underlying data or databases. It has classes and methods to retrieve and manipulate data.

The following are a few of the .NET applications that use ADO.NET to connect to a database, execute commands and retrieve data from the database.

- Windows Forms Applications.
- ASP.NET Web Applications
- Console Applications

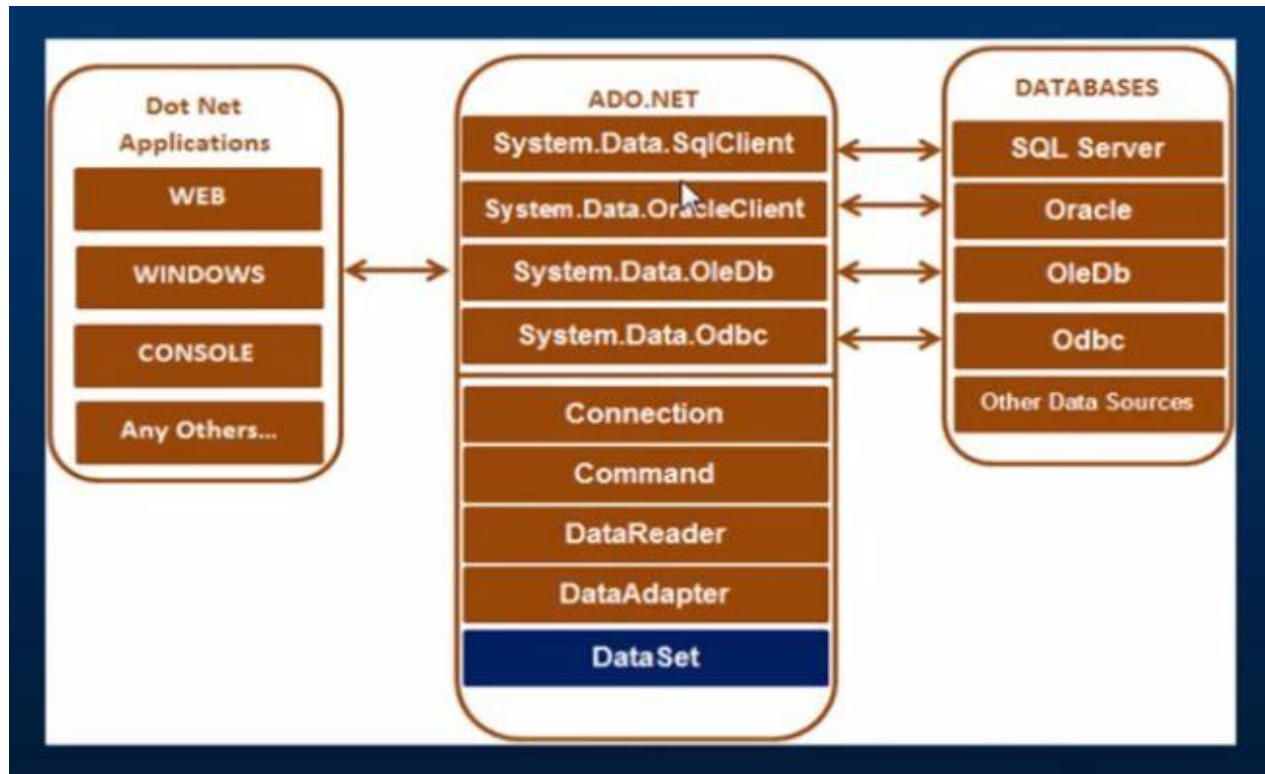
Connection Architectures-

There are the following two types of connection architectures:

1. **Connected architecture:** the application remains connected with the database throughout the processing.
2. **Disconnected architecture:** the application automatically connects/disconnects during the processing. The application uses temporary data on the application side called a **DataSet**.

Understanding ADO.NET and its class library

In following diagram, we can see that there are various types of *applications* (Web Application, Console Application, Windows Application and so on) that use **ADO.NET** to connect to different databases (SQL Server, Oracle, OleDb, ODBC, XML files and so on).



Important Classes in ADO.NET

We can also observe various classes in the preceding diagram. They are:

1. **Connection Class**
2. **Command Class**
3. **DataReader Class**
4. **DataAdapter Class**

1. Connection Class

In ADO.NET, we use these connection classes to connect to the database. These connection classes also manage transactions and connection pooling.

ADO.NET connection is an **object that provides database connectivity and the entry point to a database.**

When the connection of an object is instantiated, the constructor takes a connection string that contains the information about the database server, server type, database name, connection type, and database user credentials.

Once the connection string is passed and the connection object is created, you can establish a connection with the database.

Which namespace or provider is used for connection class?

ADO.NET provides connection to multiple providers.

Each provider has a functionality to connect with different database. Here is a list of data providers in ADO.NET and their purpose.

- ***Data Provider for SQL Server (System.Data.SqlClient).***
- ***Data Provider for MS ACCESS (System.Data.OleDb).***
- ***Data Provider for MYSQL (System.Data.Odbc).***
- ***Data Provider for ORACLE (System.Data.OracleClient).***

How to use connection class with this provider is given below-

- ***Connection object for SQL Server (SqlConnection).***
- ***Connection object for MSACCESS (OleDbConnection).***
- ***Connection object for MYSQL (OdbcConnection).***
- ***Connection object for ORACLE (OracleConnection).***

Properties of connection object

Some of commonly used properties of connection class are,

Property	Description
<i>Connection String</i>	<i>Connection string is used to establish and create connection to data source by using server name, database name, user id and password.</i>
<i>Mode</i>	<i>By mode property, we can check provider access permission.</i>
<i>Provider</i>	<i>By this property, we can get or set provider name.</i>
<i>State</i>	<i>By this property, we can check your current connection open or close before connection opening or closing</i>

Methods of connection object

Method	Description
<i>BeginTransaction</i>	<i>Begin to current transaction.</i>
<i>Cancel</i>	<i>Cancel an execution.</i>
<i>Close</i>	<i>Close method is used, when any current connection is open and finally its closed after completed execution.</i>
<i>Open</i>	<i>Open method is used, if current connection is close then before execution started. First of all You have opened connection must.</i>
<i>Execute</i>	<i>By this method it is used to execute query. Like as Statement, procedure or provider provides specific text.</i>

RollBackTransation	This method invokes, whenever you cancel any changes or any conflict occurs in the current transaction, it ends the current transaction.
CommitTransation	If current transaction execution is successfully completed, it ends the current transaction.

2. Command Class

The Command class provides methods for storing and executing SQL statements and Stored Procedures.

The command object is one of the basic components of ADO .NET.

- 1. The Command Object uses the connection object to execute SQL queries.*
- 2. The queries can be in the Form of Inline text, Stored Procedures or direct Table access.*
- 3. An important feature of Command object is that it can be used to execute queries and Stored Procedures with Parameters.*
- 4. If a select query is issued, the result set it returns is usually stored in either a DataSet or a DataReader object.*

The properties associated with SqlCommand class are shown in the Table below.

Property	Type of Access	Description
Connection	Read/Write	The SqlConnection object that is used by the command object to execute SQL queries or Stored Procedure.
CommandText	Read/Write	Represents the T-SQL Statement or the name of the Stored Procedure.
CommandType	Read/Write	This property indicates how the CommandText property should be interpreted. The possible values are: 1. Text (T-SQL Statement) 2. StoredProcedure (Stored Procedure Name) 3. TableDirect
CommandTimeout	Read/Write	This property indicates the time to wait when executing a particular command. Default Time for Execution of Command is 30 Seconds. The Command is aborted after it times out and an exception is thrown.

Now, Let us have a look at various Execute Methods that can be called from a Command Object.

Method	Description
ExecuteNonQuery	This method executes the command specifies and returns the number of rows affected.
ExecuteReader	The ExecuteReader method executes the command specified and returns an instance of SqlDataReader class.
ExecuteScalar	This method executes the command specified and returns the first column of first row of the result set. The remaining rows and column are ignored.
ExecuteXMLReader	This method executes the command specified and returns an instance of XmlReader class. This method can be used to return the result set in the form of an XML document

3. DataReader Class

The DataReader is used to retrieve data. It is used in conjunction with the Command class to execute an SQL Select statement and then access the returned rows.

A data reader provides an easy way for the programmer to read data from a database as if it were coming from a stream.

The DataReader is the solution for forward streaming data through ADO.NET.

The data reader is also called a fire hose cursor or forward read-only cursor because it moves forward through the data.

The data reader not only allows you to move forward through each record of database, but it also enables you to parse the data from each column. The DataReader class represents a data reader in ADO.NET.

DataReader Properties and Methods

DataReader properties

PROPERTY	DESCRIPTION
Depth	Indicates the depth of nesting for row
FieldCount	Returns number of columns in a row
IsClosed	Indicates whether a data reader is closed
Item	Gets the value of a column in native format
RecordsAffected	Number of row affected after a transaction

DataReader methods

METHOD	DESCRIPTION
Close	<i>Closes a DataReader object.</i>
Read	<i>Reads next record in the data reader.</i>
NextResult	Advances the data reader to the next result during batch transactions.
Getxxx	There are dozens of Getxxx methods. These methods read a specific data type value from a column. For example. GetChar will return a column value as a character and GetString as a string.

4. DataAdapter Class

The DataAdapter is used to connect DataSets to databases. The DataAdapter is most useful when using data-bound controls in Windows Forms, but it can also be used to provide an easy way to manage the connection between your application and the underlying database tables, views and Stored Procedures.

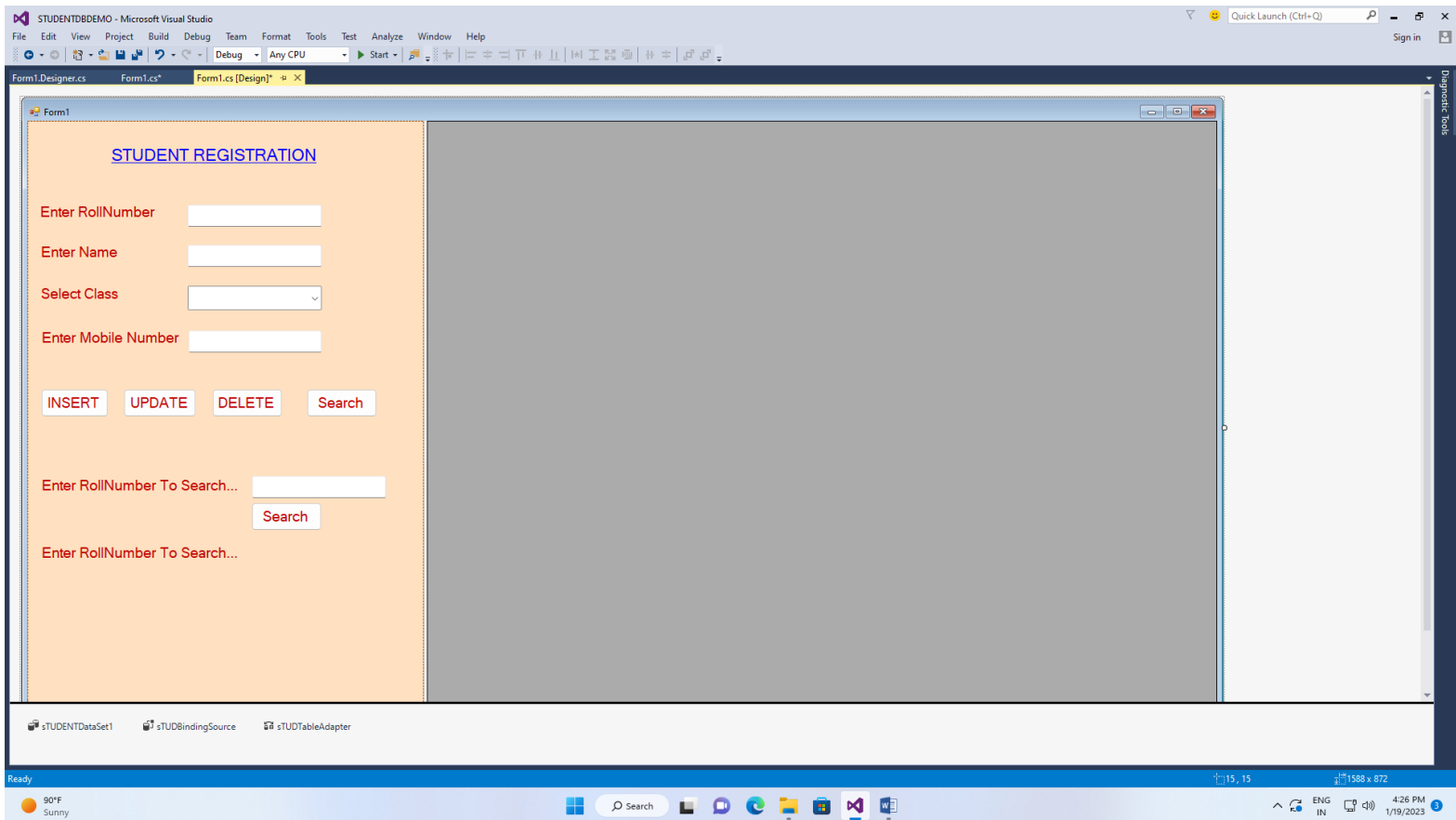
The DataAdapter works as a bridge between a DataSet and a data source to retrieve data. DataAdapter is a class that represents a set of SQL commands and a database connection. It can be used to fill the DataSet and update the data source.

Methods

Method	Description
1. Dispose(Boolean)	It is used to release the unmanaged resources used by the DataAdapter.
2. Fill(DataSet)	It is used to add rows in the DataSet to match those in the data source.
3. FillSchema(DataSet, SchemaType, String, IDataReader)	It is used to add a DataTable to the specified DataSet.
4. GetFillParameters()	It is used to get the parameters set by the user when executing an SQL SELECT statement.
5. ResetFillLoadOption()	It is used to reset FillLoadOption to its default state.
6. Update(DataSet)	It is used to call the respective INSERT, UPDATE, or DELETE statements.

Ex. To retrieve data from STUD table using DataAdapter

```
private void button5_Click(object sender, EventArgs e)
{
    try
    {
        OracleConnection con = new OracleConnection("Uid=tyecs;Password=tyecs");
        con.Open();
        string s = "select * from stud";
        OracleDataAdapter ad = new OracleDataAdapter(s,
            con); DataSet ds = new DataSet();
        ad.Fill(ds);
        dataGridView1.AutoGenerateColumns =
            true; dataGridView1.DataSource =
            ds.Tables[0];
        con.Close();
    }
    catch (Exception e1)
    {
        MessageBox.Show(e1.Message);
    }
}
```

Ex- Includes All DB Operations

```
using System.Data.OleDb;
```

```
namespace STUDENTDBDEMO
```

```
{
    public partial class Form1 : Form
    {
```

```
OleDbConnection ocn = new OleDbConnection(@"Provider=Microsoft.ACE.OLEDB.12.0;Data
Source=C:\Users\COLLEGE\Documents\Visual Studio
2015\Projects\STUDENTDBDEMO\STUDENTDBDEMO\STUDENT.accdb");
```

```
OleDbCommand ocmd = new OleDbCommand();
```

```
public Form1()
{
    InitializeComponent();
```

```
}
```

```
private void Insert_Click(object sender, EventArgs e)
{
    ocn.Open();

    ocmd.CommandText = "insert into STUD values(" + textBox1.Text + "," +
    textBox2.Text + "," + comboBox1.SelectedItem.ToString() + "," +
    textBox3.Text + ")";

    ocmd.Connection = ocn;

    int rs = ocmd.ExecuteNonQuery();

    if (rs > 0)
    {
        MessageBox.Show("Record Inserted Successfully.....");
    }

    else
        MessageBox.Show("Record Insertion Failed....");

    ocn.Close();
}
```

```
private void update_Click(object sender, EventArgs e)
{
    ocn.Open();

    ocmd.CommandText = "update STUD set
    studname='"+textBox2.Text+"',class='"+comboBox1.SelectedItem.ToString()+"',m
    obilenumber='"+textBox3.Text+"'where rollnum="+textBox1.Text;
```

```

ocmd.Connection = ocn;

int rs = ocmd.ExecuteNonQuery();
    if (rs > 0)
        MessageBox.Show("Record Updated Successfully.....");
    else
        MessageBox.Show("Record Updating Failed....");
    ocn.Close();
}

```

```

private void delete_Click(object sender, EventArgs e)
{
    ocn.Open();

    ocmd.CommandText = " delete * from STUD where rollnum=" + textBox1.Text;

    ocmd.Connection = ocn;

    int rs = ocmd.ExecuteNonQuery();
        if (rs > 0)
            MessageBox.Show("Record Deleted Successfully.....");
        else
            MessageBox.Show("Record Deletion Failed....");
        ocn.Close();
}

```

```

private void Search_Click(object sender, EventArgs e)
{
    ocn.Open();

    string str = "Data of Roll num " + textBox4.Text + " is....";
}

```

```

ocmd.CommandText = "select * from STUD where rollnum=" +
int.Parse(textBox4.Text);

ocmd.Connection = ocn;

OleDbDataReader dr = ocmd.ExecuteReader();

if (dr.Read())
{
    str = str + "\tNAME=" + dr[1] + "\tCLASS" + dr[2] + "\tMOBILENUMBER=" +
dr[3];
}

label7.Text = str;

dr.Close();

ocn.Close();
}

```

STUD Table in STUDENT.accdb Database

rollnum	studname	class	mobilenum
101	AAA	B.C.A.-I	1212121212
102	asdas	B.Sc(CS)-II	1234567890
103	shiv	B.Sc(ECS)-II	1212312312
105	ccc	B.Sc(ECS)-II	1212313112
105	ccc	B.Sc(ECS)-II	1212313112
105	ccc	B.Sc(ECS)-II	1212313112
106	xxxx	B.Sc(ECS)-II	1111111111
111	AAAA	M.Sc(CS)-I	1211211213
122	SSS	M.C.A.-III	1212121212
444	QQQ	B.C.A.-III	1234512345
555	FFF	B.Sc(CS)-I	1234123456
666	GGG	B.Sc(CS)-III	1231231234
777	www	B.C.A.-II	1234554321
888	mmm	M.Sc(CS)-I	1245612456
123	sdfg	B.Sc(ECS)-II	123124124
*	NULL	NULL	NULL